

ISLAMIC UNIVERSITY OF TECHNOLOGY

Board Bazar, Gazipur, Dhaka.



Assignment : Main Memory Management in Real-World Operating Systems Operating Systems : CSE 4501

<u>Student ID: 210042112</u>

Fifth Semester
Department of Computer Science and Engineering
BSc. in Software Engineering

April 30, 2025

Abstract

Operating systems must manage main memory efficiently to maximize performance and resource utilization. In this assignment, choose two modern operating systems (e.g., Windows, Linux, macOS, Android, etc.) and perform a comparative analysis of how they handle main memory management.

Contents

1	Preface	4
2	Design Goals Influencing Memory Managemen	4
2.1	macOS	4
2.2	Android	4
3	Allocation and Deallocation Strategies	4
3.1	macOS	4
3.2	Android	4
3.3	Comparison	5
4	Memory Protection Mechanisms	5
4.1	macOS	5
4.2	Android	5
4.3	Comparison	5
5	Contiguous vs. Non-Contiguous Memory Allocation	6
5.1	macOS	6
5.2	Android	6
5.3	Comparison	6
6	Fragmentation Handling	6
6.1	macOS	6
6.2	Android	6
6.3	Comparison	7
7	Trade-Offs	7
7.1	macOS	7
7.2	Android	7
8	Theoretical Concepts	7
9	Real-World Approaches	8
10	References	9

1 Preface

Memory management is a cornerstone of operating system design, ensuring efficient resource utilization and system stability. This analysis compares the memory management strategies of macOS (version 15 Sequoia, 2024) and Android (version 15, 2024).

2 Design Goals Influencing Memory Management

2.1 macOS

- **Performance and Responsiveness:** macOS is optimized for Apple's ecosystem, prioritizing smooth performance for creative and productivity applications on resource-constrained devices like MacBooks.
- **Security:** macOS emphasizes robust memory protection to safeguard user data and prevent unauthorized access, aligning with Apple's focus on privacy.
- **Integration:** Memory management is tightly integrated with Apple's hardware, enabling optimizations like unified memory architecture (UMA) on Apple Silicon.

2.2 Android

- **Flexibility:** Android runs on diverse hardware, from low-end to flagship devices, requiring adaptable memory management to support varying RAM capacities.
- **Multitasking:** Android prioritizes efficient multitasking for apps, keeping background processes in memory to enable quick switching.
- **Battery Efficiency:** Memory management is designed to minimize power consumption, critical for mobile devices.

3 Allocation and Deallocation Strategies

3.1 macOS

- **XNU Kernel Allocator:** macOS uses the XNU kernel's zone allocator for kernel memory, pre-allocating memory pools for specific object types to reduce allocation overhead. The Mach Virtual Memory (VM) system manages user-space memory.
- **Unified Memory Architecture:** On Apple Silicon, macOS employs UMA, where CPU and GPU share a single memory pool, reducing duplication and improving allocation efficiency.
- **Memory Compression:** macOS compresses inactive pages to free up RAM without swapping to disk, using the *Compressor* subsystem.
- **Deallocation:** The Mach VM system deallocates memory lazily, reclaiming pages via the *vm_pageout* daemon when memory pressure increases. Garbage collection is absent, relying on Automatic Reference Counting (ARC) for user-space apps.

3.2 Android

- **ART and Dalvik VM:** Android's Android Runtime (ART) and Dalvik virtual machine use a managed memory environment. Memory is allocated via heap-based allocation for apps, with paging and memory-mapping (mmap) for efficiency.
- **Generational Heap:** Android's heap is generational, dividing allocations into Young, Old, and Permanent generations to optimize garbage collection based on object lifespan.

- **Garbage Collection:** Android relies on garbage collection to deallocate memory, triggered when heap generations fill up. The garbage collector reclaims memory from unreferenced objects, though frequent collections can impact performance.
- **Zygote Process:** Android forks app processes from the Zygote process, preloading framework code and resources to share memory pages across apps, reducing allocation overhead.

3.3 Comparison

macOS's zone allocator and UMA prioritize performance and hardware integration, while Android's generational heap and Zygote optimize for multitasking and resource sharing on diverse devices. macOS's memory compression reduces disk I/O, whereas Android's garbage collection introduces performance variability but simplifies deallocation.

4 Memory Protection Mechanisms

4.1 macOS

- **Page-Based Protection:** macOS uses hardware-enforced page tables to isolate processes, marking pages with access permissions (e.g., read-only, executable). The XNU kernel enforces user-kernel separation.
- **Address Space Layout Randomization (ASLR):** macOS randomizes memory addresses to mitigate exploits.
- **Code Signing and Sandboxing:** macOS enforces code signing and app sandboxing, restricting memory access to authorized processes.
- **Memory Tagging:** On Apple Silicon, macOS uses pointer authentication and memory tagging to prevent memory corruption and buffer overflows.

4.2 Android

- **Page-Based Protection:** Android employs page tables to isolate app processes, with ART/Dalvik enforcing memory boundaries. The Linux kernel under Android ensures user-kernel isolation.
- **ASLR:** Android uses ASLR to randomize memory layouts, enhancing security.
- **SELinux:** Android leverages Security-Enhanced Linux (SELinux) to enforce mandatory access controls, restricting memory access at the kernel level.
- **Scoped Storage:** Android's scoped storage limits app access to memory-mapped files, reducing unauthorized memory interactions.

4.3 Comparison

Both systems use page-based protection and ASLR, but macOS's code signing, sandboxing, and memory tagging provide deeper security tailored to Apple's ecosystem. Android's SELinux and scoped storage focus on flexibility and compatibility across diverse hardware, though they may be less stringent than macOS's mechanisms.

5 Contiguous vs. Non-Contiguous Memory Allocation

5.1 macOS

- **Non-Contiguous Allocation:** macOS primarily uses non-contiguous allocation via paging. The Mach VM system maps physical pages to virtual addresses, allowing scattered physical memory to appear contiguous to processes.
- **Contiguous Allocation (Limited):** Contiguous allocation is used for specific kernel tasks or hardware drivers (e.g., GPU buffers), facilitated by the IOKit framework, but is rare in user space.
- **Large Pages:** macOS supports large pages (e.g., 2 MB) to reduce Translation Lookaside Buffer (TLB) misses, improving performance for memory-intensive tasks.

5.2 Android

- **Non-Contiguous Allocation:** Android relies on paging and memory-mapping (mmap) for non-contiguous allocation. ART/Dalvik allocate memory in non-contiguous blocks, mapped to virtual address spaces.
- **Contiguous Allocation (Rare):** Contiguous allocation is used for specific kernel drivers (e.g., DMA for hardware), but Android's Linux kernel avoids it in user space to maximize flexibility.
- **Page-Based Heap:** Android's heap is divided into fixed-size pages, ensuring non-contiguous allocation to minimize fragmentation.

5.3 Comparison

Both systems favor non-contiguous allocation through paging, aligning with modern hardware capabilities. macOS's large page support optimizes performance, while Android's page-based heap ensures compatibility across devices. Contiguous allocation is minimal in both, reserved for specialized kernel tasks.

6 Fragmentation Handling

6.1 macOS

- **Internal Fragmentation:** The zone allocator minimizes internal fragmentation by allocating fixed-size memory pools for kernel objects. In user space, ARC reduces memory leaks, indirectly mitigating fragmentation.
- **External Fragmentation:** macOS uses page compaction to consolidate free memory pages, reducing external fragmentation. Memory compression further alleviates pressure by shrinking inactive pages.
- **Unified Memory:** UMA on Apple Silicon reduces fragmentation by sharing memory across CPU and GPU, eliminating redundant allocations.

6.2 Android

- **Internal Fragmentation:** Android's generational heap can cause internal fragmentation when small allocations share pages with active objects, preventing page reclamation. The garbage collector mitigates this by freeing unused objects.
- **External Fragmentation:** Android reduces external fragmentation by returning unused pages to the kernel via madvise after garbage collection. However, the Dalvik heap does not compact, relying on paging to manage scattered free memory.
- **Memory Pools:** Android uses memory pools for specific allocations (e.g., graphics buffers), reducing fragmentation for fixed-size objects.

6.3 Comparison

macOS's zone allocator and UMA effectively minimize both internal and external fragmentation, leveraging Apple's hardware control. Android's reliance on garbage collection and non-compacting heap leads to higher fragmentation risk, mitigated by paging and memory pools. macOS's compaction is more aggressive than Android's madvise-based approach.

7 Trade-Offs

7.1 macOS

- **Performance vs. Complexity:** UMA and memory compression enhance performance but increase system complexity, requiring tight hardware-software integration.
- **Security vs. Overhead:** Code signing, sandboxing, and memory tagging improve security but add computational overhead, impacting performance on older hardware.
- **Ecosystem Control vs. Flexibility:** macOS's optimized memory management relies on Apple's hardware, limiting portability to non-Apple devices.

7.2 Android

- **Flexibility vs. Efficiency:** Android's adaptable memory management supports diverse hardware but sacrifices efficiency compared to macOS's tailored approach.
- **Performance vs. Battery Life:** Garbage collection enables efficient memory reuse but can degrade performance and consume battery during frequent collections.
- **Multitasking vs. Stability:** Keeping apps in memory via Zygote improves multitasking but risks low-memory conditions, triggering the Low Memory Killer (LMK) to terminate processes.

8 Theoretical Concepts

1. **Virtual Memory:** Both macOS and Android implement virtual memory via paging, providing processes with isolated address spaces. macOS's Mach VM and Android's ART/Dalvik align with this principle, using demand paging to optimize resource usage.
2. **Page Replacement:** macOS uses a global LRU-based algorithm for page replacement, while Android employs a per-process working set model, reflecting theoretical trade-offs between global and local replacement strategies.
3. **Fragmentation:** macOS's compaction and Android's paging address external fragmentation, while memory pools and zone allocators tackle internal fragmentation, consistent with theoretical defragmentation techniques.
4. **Memory Protection:** Both systems use hardware-enforced page tables and ASLR, aligning with theoretical isolation and security principles.
5. **Allocation Algorithms:** macOS's zone allocator resembles theoretical pool-based allocation, while Android's generational heap mirrors generational garbage collection models.

9 Real-World Approaches

- macOS’s UMA is a practical innovation, extending theoretical shared memory concepts to CPU-GPU integration, absent from traditional models.
- Android’s Zygote process optimizes memory sharing across apps, a pragmatic approach not typically covered in theoretical discussions.
- Android’s Low Memory Killer, similar to Linux’s OOM killer, addresses memory exhaustion in real-time, a solution absent from theoretical frameworks.

macOS and Android adopt memory management strategies tailored to their design goals. macOS leverages UMA, memory compression, and robust security mechanisms to deliver high performance and stability within Apple’s ecosystem. Android prioritizes flexibility and multitasking, using a generational heap, Zygote, and garbage collection to support diverse hardware. Both align with theoretical concepts like virtual memory and paging but introduce innovations like UMA and Zygote to address real-world challenges. macOS trades flexibility for efficiency, while Android balances versatility with performance, reflecting their distinct target environments.

10 References

1. [Apple vs Android RAM management: Who does it better?](#)
2. [Android memory management](#)
3. [Contiguous Memory Allocation in Operating System](#)
4. [Memory Management Strategies in Operating System](#)
5. [Memory Management in Operating System](#)
6. [Memory Management in OS: Contiguous, Swapping, Fragmentation](#)